LAPORAN PRATIKUM DATA ENGINEERING AND BIG DATA SYSTEMS

PERTEMUAN 6

OLEH

Nama : AYUL HUSNAINI

Nim : 105841110423

Kelas : 5 AI-A

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

1. Persiapan Data dan Perbandingan Tiga Regressor

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
import numpy as np

# Persiapan Data    "Persiapan": Unknown word.
db = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    db.data, db.target, test_size=0.2, random_state=42
)

print("=" * 60)
print("PERBANDINGAN TIGA MODEL REGRESI PADA DATASET DIABETES")    "PERBANDINGAN": Unknown word.
print("=" * 60)

# Model 1: Decision Tree Regressor
dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X_train, y_train)
dt_pred = dt_reg.predict(X_test)
dt_rmse = np.sqrt(mean_squared_error(y_test, dt_pred))
dt_r2 = r2_score(y_test, dt_pred)

print("\n1. Decision Tree Regressor")
print(f"   RMSE: {dt_rmse:.4f}")
print(f"   R²: {dt_r2:.4f}")

# Model 2: Random Forest Regressor
rf_reg = RandomForestRegressor(n_estimators=100, random_state=42)
rf_reg.fit(X_train, y_train)
rf_pred = rf_reg.predict(X_test)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
rf_r2 = r2_score(y_test, rf_pred)

print("\n2. Random Forest Regressor (100 pohon)")    "pohon": Unknown word.
print(f"   RMSE: {rf_rmse:.4f}")
print(f"   R²: {rf_r2:.4f}")
```

```python
# Model 3: Gradient Boosting Regressor
gb_reg = GradientBoostingRegressor(n_estimators=100, random_state=42)
gb_reg.fit(X_train, y_train)
gb_pred = gb_reg.predict(X_test)
gb_rmse = np.sqrt(mean_squared_error(y_test, gb_pred))
gb_r2 = r2_score(y_test, gb_pred)

print("\n3. Gradient Boosting Regressor (100 estimators)")
print(f"   RMSE: {gb_rmse:.4f}")
print(f"   R²: {gb_r2:.4f}")

# Ringkasan Perbandingan    "Ringkasan": Unknown word.
print("\n" + "=" * 60)
print("RINGKASAN PERBANDINGAN")    "RINGKASAN": Unknown word.
print("=" * 60)
results_df = pd.DataFrame({
    'Model': ['Decision Tree', 'Random Forest', 'Gradient Boosting'],
    'RMSE': [dt_rmse, rf_rmse, gb_rmse],
    'R²': [dt_r2, rf_r2, gb_r2]
})
print(results_df.to_string(index=False))
print(f"\nModel terbaik: {results_df.loc[results_df['R²'].idxmax(), 'Model']}")    "terbaik": Unknown word.
print("(R² paling tinggi menunjukkan fit yang lebih baik)")    "tinggi": Unknown word.
```

```
============================================================
PERBANDINGAN TIGA MODEL REGRESI PADA DATASET DIABETES
============================================================

1. Decision Tree Regressor
   RMSE: 70.5464
   R²: 0.0607

2. Random Forest Regressor (100 pohon)
   RMSE: 54.3324
   R²: 0.4428

3. Gradient Boosting Regressor (100 estimators)
   RMSE: 53.8371
   R²: 0.4529

============================================================
RINGKASAN PERBANDINGAN
============================================================
            Model      RMSE       R²
    Decision Tree 70.546423 0.060654
    Random Forest 54.332408 0.442823
Gradient Boosting 53.837131 0.452934

Model terbaik: Gradient Boosting
(R² paling tinggi menunjukkan fit yang lebih baik)
```

- from sklearn.datasets import load_diabetes

  Digunakan untuk:

  Memanggil fungsi load_diabetes dari scikit-learn untuk mengambil dataset diabetes.
- from sklearn.model_selection import train_test_split

  Digunakan untuk:

  Memanggil fungsi train_test_split untuk membagi data menjadi data latih dan data uji.
- from sklearn.tree import DecisionTreeRegressor

  Digunakan untuk:

  Memanggil kelas DecisionTreeRegressor untuk membuat dan melatih model pohon keputusan regresi.
- from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

  Digunakan untuk: Memanggil

  kelas RandomForestRegressor dan GradientBoostingRegressor untuk membuat dan melatih model ensemble.
- from sklearn.metrics import mean_squared_error, r2_score

Digunakan untuk:

Memanggil fungsi mean_squared_error dan r2**_score** untuk mengukur performa model regresi.

- import pandas as pd

  Digunakan untuk:

  Memanggil library Pandas untuk manipulasi data tabel, seperti membuat DataFrame hasil evaluasi.

- import numpy as np

  Digunakan untuk:

  Memanggil library NumPy untuk operasi matematika, seperti menghitung akar kuadrat RMSE.

- db = load_diabetes()

  X_train, X_test, y_train, y_test = train_test_split(
  db.data, db.target, test_size=0.2, random_state=42)

  Digunakan untuk:

  Memuat data diabetes dan membaginya menjadi data latih (80%) dan data uji (20%).

- dt_reg = DecisionTreeRegressor(random_state=42)

  dt_reg.fit(X_train, y_train)

  dt_pred = dt_reg.predict(X_test)

  dt_rmse = np.sqrt(mean_squared_error(y_test, dt_pred))

  dt_r2 = r2_score(y_test, dt_pred)

  Digunakan untuk:

  Membuat, melatih, dan mengevaluasi model Decision Tree pada data uji.

- f_reg = RandomForestRegressor(n_estimators=100, random_state=42)

  rf_reg.fit(X_train, y_train)

  rf_pred = rf_reg.predict(X_test)

  rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

  rf_r2 = r2_score(y_test, rf_pred)

Digunakan untuk:

Membuat, melatih, dan mengevaluasi model Random Forest dengan 100 pohon pada data uji.

- gb_reg = GradientBoostingRegressor(n_estimators=100, random_state=42)
  gb_reg.fit(X_train, y_train)
  gb_pred = gb_reg.predict(X_test)
  gb_rmse = np.sqrt(mean_squared_error(y_test, gb_pred))
  gb_r2 = r2_score(y_test, gb_pred)
  Digunakan untuk:
  Membuat, melatih, dan mengevaluasi model Gradient Boosting dengan 100 estimator pada data uji.
- results_df = pd.DataFrame({
  'Model': ['Decision Tree', 'Random Forest', 'Gradient Boosting'],
  'RMSE': [dt_rmse, rf_rmse, gb_rmse],
  'R²': [dt_r2, rf_r2, gb_r2]})
  Digunakan untuk:
  Membuat tabel ringkasan hasil evaluasi tiga model menggunakan DataFrame Pandas.

2. Menampilkan 10 Fitur Terpenting dari Random Forest

- feature_importance = pd.Series(
  rf_reg.feature_importances_,
  index=db.feature_names
).sort_values(ascending=False)
  Digunakan untuk:
  Mengambil skor pentingnya fitur dari model Random Forest dan mengurutkannya dari yang paling berpengaruh.
- top_10 = feature_importance.head(10)
  for i, (feature, importance) in enumerate(top_10.items(), 1):
  bar = '[redacted]' * int(importance * 100)
  print(f"{i:2d}. {feature:15s} {importance:.4f} {bar}")

Digunakan untuk:

Menampilkan 10 fitur terpenting beserta nilai dan visualisasi bar sederhana agar mudah dipahami.

3. Eksperimen Variasi Parameter n_estimators
   - import time

     Digunakan untuk:

     Memanggil modul time untuk mengukur lama waktu pelatihan model.
   - n_estimators_list = [10, 50, 100, 200]

     results_list = []

     Digunakan untuk:

     Menyiapkan daftar jumlah pohon yang akan diuji dan wadah hasil eksperimen
   - for n in n_estimators_list:

     ```
     start_time = time.time()
     rf_temp = RandomForestRegressor(n_estimators=n, random_state=42)
     rf_temp.fit(X_train, y_train)
     pred_temp = rf_temp.predict(X_test)
     elapsed = time.time() - start_time
     rmse = np.sqrt(mean_squared_error(y_test, pred_temp))
     r2 = r2_score(y_test, pred_temp)
     results_list.append({
         'n_estimators': n,
         'RMSE': rmse,
         'R²': r2,
         'Waktu (detik)': elapsed
     })
     ```

     Digunakan untuk:

     Melatih Random Forest dengan berbagai jumlah pohon, mengukur akurasi dan waktu, lalu menyimpan hasilnya.
   - results_n_df = pd.DataFrame(results_list)

Digunakan untuk:

Membuat tabel ringkasan hasil eksperimen variasi n_estimators menggunakan DataFrame Pandas.